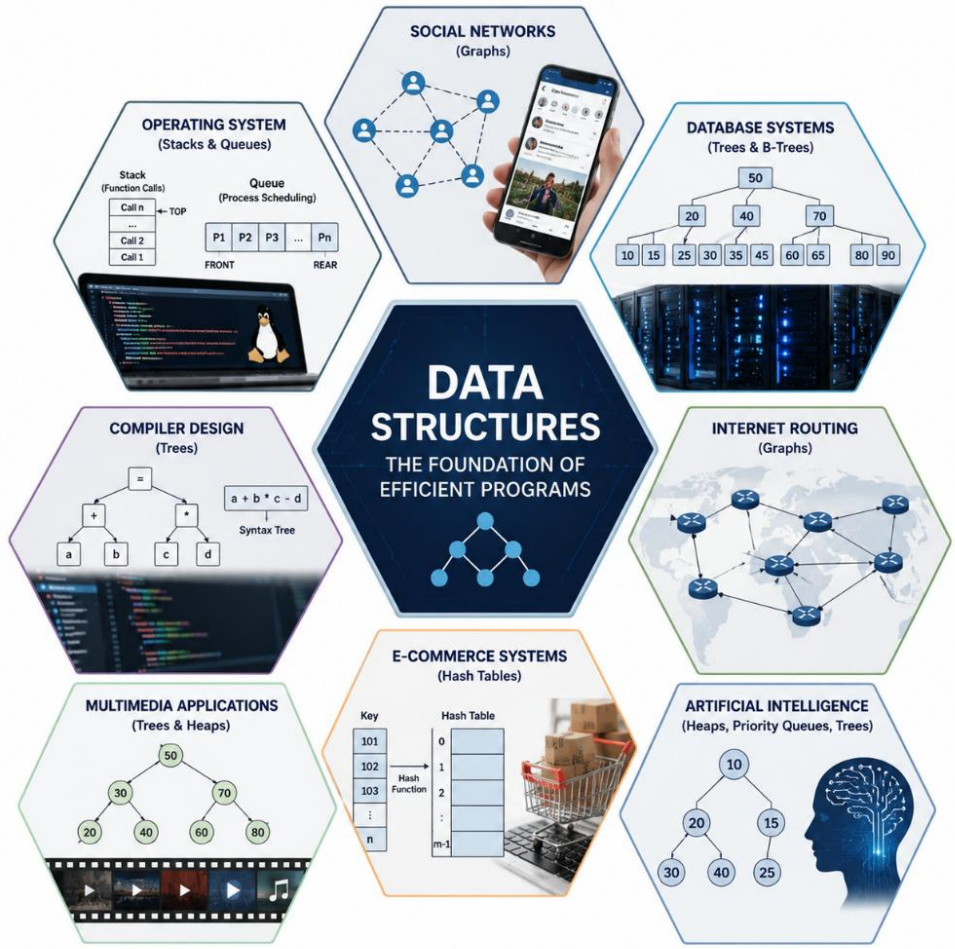
Data Structure & Algorithm Lab

ECE2132



Department of Electronics & Communication Engineering

By:

Dr. D.V. Prashant
Dr. Kamal Kishore Upadhyay

Manipal University Jaipur,

Dehmi Kalan, Off Jaipur-Ajmer Expressway, Jaipur, (Raj.)

Rajasthan 303007

MANIPAL UNIVERSITY
JAIPUR
(University under Section 2(f) of the UGC Act)

(AY 2026-27)

EXPERIMENT NO. 7

WRITE A PROGRAM TO IMPLEMENT INSERTION SORT FOR SORTING AN ARRAY OF INTEGERS AND ANALYSE ITS PERFORMANCE

AIM: To study and implement the Insertion Sort algorithm in C++ for sorting an array of integers in ascending order and analyse its time and space complexity.

SOFTWARE USED: Visual Studio Code / Code: Blocks / Dev-C++, GCC/G++ Compiler. Operating System: Windows/Linux

THEORY: Sorting is the process of arranging data elements in a particular order. In this experiment, the elements of an integer array are arranged in ascending order using the Insertion Sort algorithm.

Insertion Sort is a simple comparison-based sorting algorithm. It divides the array into a sorted portion and an unsorted portion. Initially, the first element is considered sorted.

One element is selected from the unsorted portion as the key. The key is compared with the elements in the sorted portion. Elements greater than the key are shifted one position to the right, and the key is inserted at its correct position.

This process is repeated until all elements are sorted. Insertion Sort is an in-place and stable sorting algorithm and is suitable for small or nearly sorted arrays.

Its best-case time complexity is O(n), while the average and worst-case time complexity is $O(n^2)$. The space complexity is O(1).

Insertion Sort

Insertion Sort is a simple comparison-based sorting algorithm. It works in a manner similar to arranging playing cards in your hand.

The algorithm divides the array into two parts:

Sorted part – contains elements that are already arranged in ascending order.

Unsorted part – contains the remaining elements that are yet to be sorted.

Initially, the first element is sorted. The next element is selected as the key and compared with the elements in the sorted portion. Elements greater than the key are shifted one position to the right, and the key is inserted into its correct position.

Example

Consider the array:

A = [5, 2, 4, 6, 1, 3]

Initially:

Sorted: [5]

Unsorted: [2, 4, 6, 1, 3]

Select 2 as the key:

[5, 2, 4, 6, 1, 3]

Since 5 > 2, shift 5 to the right and insert 2:

[2, 5, 4, 6, 1, 3]

Select 4 as the key:

[2, 5, 4, 6, 1, 3]

Shift 5 to the right:

[2, 4, 5, 6, 1, 3]

Similarly, the remaining elements are inserted into their correct positions.

Final sorted array:

[1, 2, 3, 4, 5, 6]

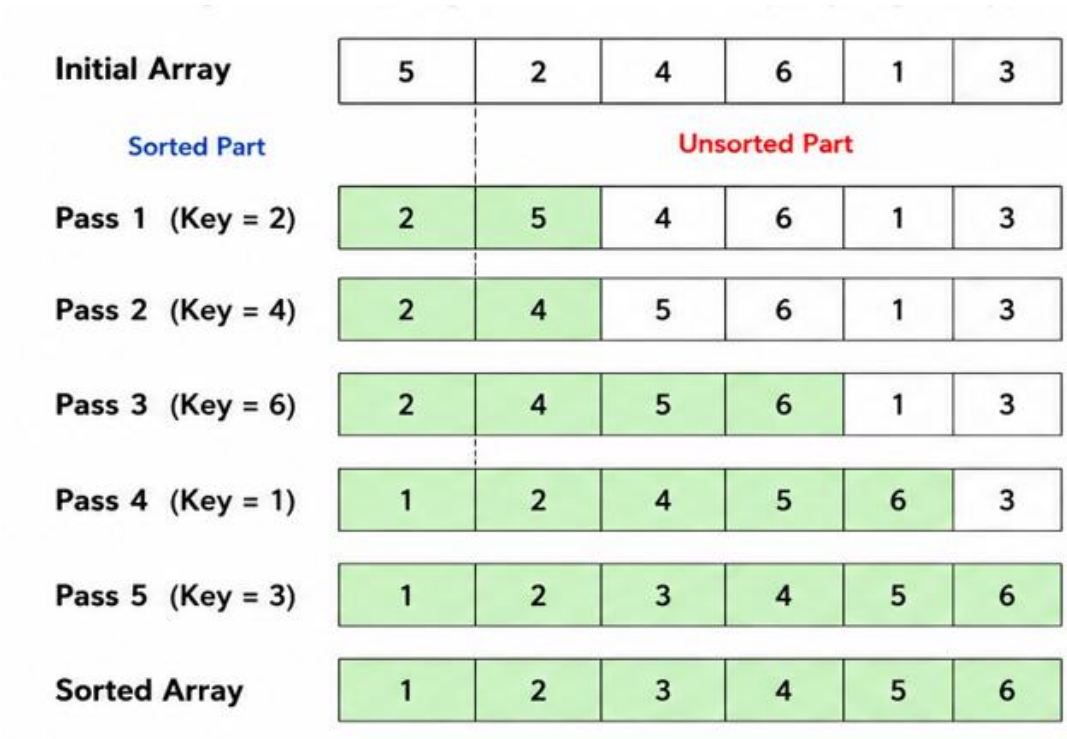


Figure. 1: Working of Insertion Sort

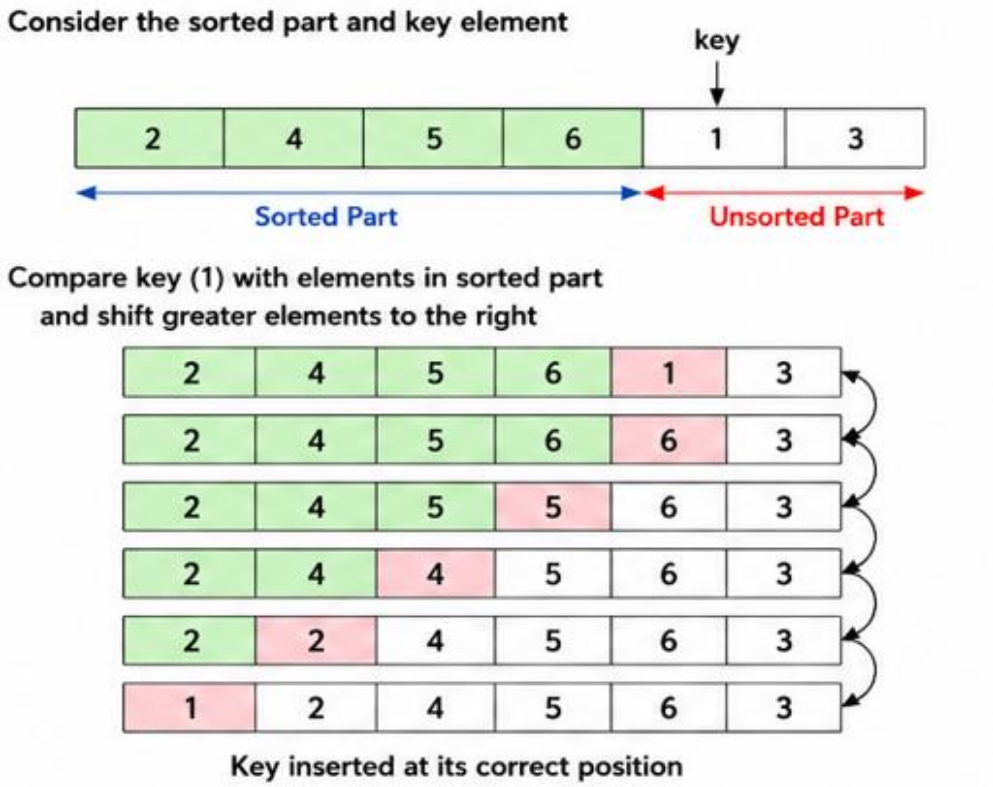


Figure. 2: Insertion of Key Elements

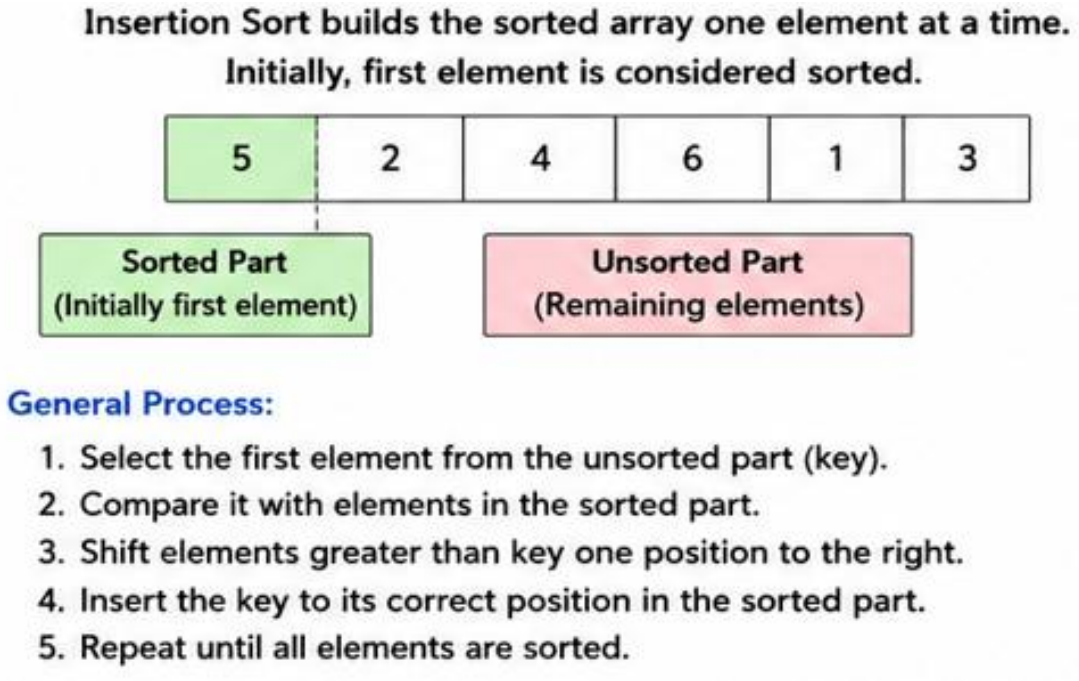


Figure. 3: Concept of Insertion Sort

Feature	Insertion Sort	Bubble Sort	Selection Sort
Type	Comparison Sort	Comparison Sort	Comparison Sort
Best Case Time Complexity	O(n)	O(n)	$O(n^2)$
Average Case Time Complexity	$O(n^2)$	$O(n^2)$	$O(n^2)$
Worst Case Time Complexity	$O(n^2)$	$O(n^2)$	$O(n^2)$
Space Complexity	O(1)	O(1)	O(1)
Stability	Stable	Stable	Unstable
In-Place	Yes	Yes	Yes

Figure. 4: Comparison with Other Sorting Algorithms

ALOGRITHM:

1. Start.
2. Declare an integer array A and enter the number of elements n.
3. Enter the elements of the array.
4. Start from the second element of the array.
5. Store the current element in key.
6. Compare key with the elements in the sorted portion of the array.
7. If an element is greater than key, shift that element one position to the right.
8. Continue shifting until the correct position for key is found.
9. Insert key into its correct position.
10. Repeat the process for all elements.
11. Display the sorted array.
12. Analyse the performance of the algorithm.
13. Stop.

PROGRAM (C++):

```
#include <iostream>
using namespace std;

// Function to perform Insertion Sort
void insertionSort(int arr[], int n)
{
    for (int i = 1; i < n; i++)
    {
        int key = arr[i];
        int j = i - 1;

        // Shift elements greater than key
        // one position to the right
        while (j >= 0 && arr[j] > key)
        {
            arr[j + 1] = arr[j];
            j--;
        }

        // Insert key at its correct position
        arr[j + 1] = key;
    }
}

// Function to display the array
void displayArray(int arr[], int n)
{
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
}

int main()
{
    int n;

    cout << "Enter number of elements: ";
    cin >> n;

    int arr[n];

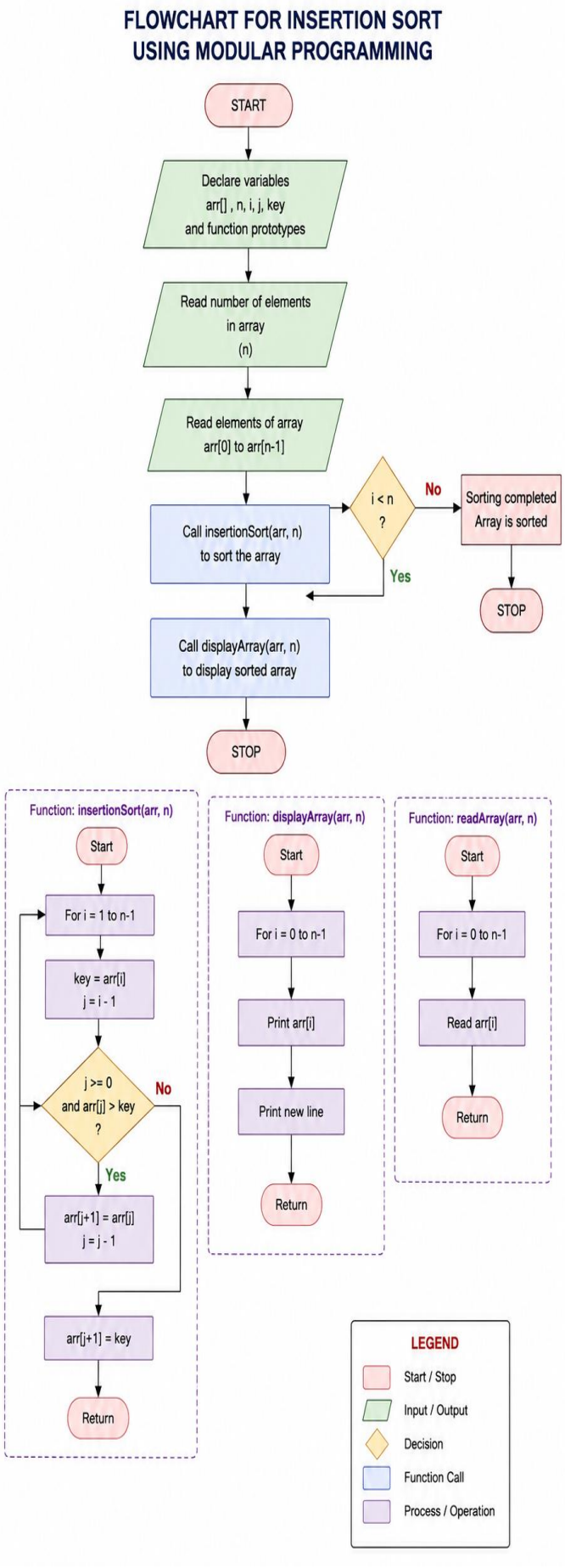
    cout << "Enter elements of the array:" << endl;
    for (int i = 0; i < n; i++)
    {
        cin >> arr[i];
    }

    cout << "Original Array: ";
    displayArray(arr, n);

    insertionSort(arr, n);

    cout << "Sorted Array: ";
    displayArray(arr, n);

    return 0;
}
```



EXPLANATION OF THE PROGRAM:

Statement / Function	Explanation
int arr[n]	Declares an integer array containing n elements.
insertionSort(arr, n)	Calls the function that performs insertion sort.
for (int i = 1; i < n; i++)	Processes each element starting from the second element.
int key = arr[i]	Stores the current element that needs to be inserted into the sorted portion.
int j = i - 1	Starts comparison from the element immediately before the key.
arr[j] > key	Checks whether the current sorted element is greater than the key.
arr[j + 1] = arr[j]	Shifts the larger element one position to the right.
j--	Moves to the previous element for further comparison.
arr[j + 1] = key	Places the key at its correct position.
displayArray()	Displays the elements of the array.
return 0	Terminates the program successfully.

SAMPLE INPUT 1:

Enter number of elements:

6

Enter elements of the array:

5 2 4 6 1 3

CALCULATION:

Initial array:

5 2 4 6 1 3

Pass 1

Key = 2

5 2 4 6 1 3

5 is greater than 2, so shift 5:

2 5 4 6 1 3

Pass 2

Key = 4

2 5 4 6 1 3

5 is greater than 4, so shift 5:

2 4 5 6 1 3

Pass 3

Key = 6

6 is already greater than 5.

2 4 5 6 1 3

Pass 4

Key = 1

Elements 6, 5, 4 and 2 are shifted to the right.

1 2 4 5 6 3

Pass 5

Key = 3

Elements 6, 5 and 4 are shifted to the right.

1 2 3 4 5 6

Therefore, the final sorted array is:

1 2 3 4 5 6

Sample Output:

Enter number of elements: 6

Enter elements of the array:

5 2 4 6 1 3

Original Array: 5 2 4 6 1 3

Sorted Array: 1 2 3 4 5 6

PERFORMANCE ANALYSIS:

Insertion Sort performance depends on the initial arrangement of the elements in the array.

Best Case

The best case occurs when the array is already sorted.

Example:

1 2 3 4 5 6

In this case, every element is already in its correct position and only a small number of comparisons are required.

Time Complexity = O(n)

Average Case

In the average case, the elements are in a random order. The algorithm performs shifting and comparisons for a significant number of elements.

Time Complexity = $O(n^2)$

Worst Case

The worst case occurs when the array is sorted in reverse order.

Example:

6 5 4 3 2 1

Every new element must be compared with and shifted past all previously sorted elements.

Time Complexity = $O(n^2)$

TIME COMPLEXITY:

Best Case $O(n)$

Average Case $O(n^2)$

Worst Case $O(n^2)$

For an array containing n elements:

Best case: approximately n - 1 comparisons.

Average case: approximately $n^2/4$ comparisons/shifts.

Worst case: approximately n(n - 1)/2 comparisons/shifts.

Therefore, the overall worst-case time complexity of Insertion Sort is:

$O(n^2)$

Space Complexity

Insertion Sort is an in-place sorting algorithm because it does not require a separate array for sorting.

The algorithm uses only a few additional variables such as key and j.

Therefore:

Space Complexity = O(1)

CHARACTERISTICS OF INSERTION SORT:

1. It is a comparison-based sorting algorithm.
2. It is an in-place sorting algorithm.
3. It is a stable sorting algorithm.
4. It performs well for small datasets.
5. It performs efficiently when the array is already or nearly sorted.
6. Its worst-case time complexity is $O(n^2)$.
7. It does not require additional array storage.

IMPLEMENTATION OF CODE

```cpp
#include <iostream>
using namespace std;

// Function to perform Insertion Sort
void insertionSort(int arr[], int n)
{
    for (int i = 1; i < n; i++)
    {
        int key = arr[i];
        int j = i - 1;

        // Shift elements greater than key
        // one position to the right
        while (j >= 0 && arr[j] > key)
        {
            arr[j + 1] = arr[j];
            j--;
        }

        // Insert key at its correct position
        arr[j + 1] = key;
    }
}

// Function to display the array
void displayArray(int arr[], int n)
{
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
}

int main()
{
    int n;

    cout << "Enter number of elements: ";
    cin >> n;

    int arr[n];

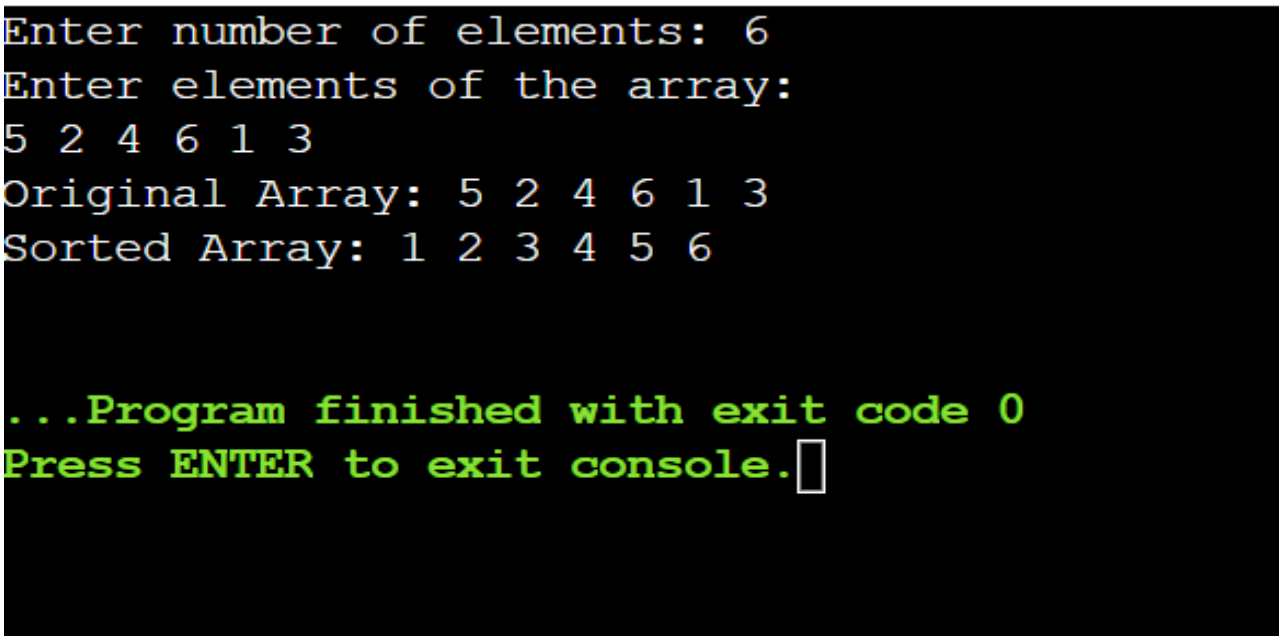
    cout << "Enter elements of the array:" << endl;
    for (int i = 0; i < n; i++)
    {
        cin >> arr[i];
    }

    cout << "Original Array: ";
    displayArray(arr, n);

    insertionSort(arr, n);

    cout << "Sorted Array: ";
    displayArray(arr, n);

    return 0;
}
```



APPLICATIONS: Insertion Sort can be used in: Sorting small datasets, Sorting data that is nearly sorted, Online sorting where elements are received one at a time, Maintaining a sorted list while new elements are inserted, educational demonstrations of sorting algorithms, Situations where a simple and low-memory sorting method is required.

RESULT: The program was successfully executed. It demonstrated the implementation of Insertion Sort in C++ for sorting an array of integers in ascending order. The performance of the algorithm was analyzed, and its best-case, average-case, worst-case time complexities and space complexity were studied.

ASSIGNMENT:

- WAP to implement Insertion Sort for sorting an array in descending order.
- WAP to implement Insertion Sort using separate functions for input, sorting, and display.
- WAP to count the number of comparisons performed by Insertion Sort.
- WAP to count the number of shifts performed during Insertion Sort.
- Compare the performance of Insertion Sort for sorted, reverse-sorted, and randomly arranged arrays.
- WAP to implement Insertion Sort for an array of floating-point numbers.
- WAP to implement Insertion Sort and find the position of a particular element after sorting.